

Community Health Platform

1. Problem Statement

Millions of people in underserved communities lack easy access to healthcare resources, doctor availability, and reliable medical guidance. Patients struggle to book appointments, track health records, or find nearby clinics — often relying on word-of-mouth or outdated directories. Meanwhile, healthcare providers lack a simple digital channel to reach and manage their local patient base.

Why It Matters

Early access to healthcare information and booking tools reduces emergency room overload, improves treatment outcomes, and empowers patients to take ownership of their health. A community-centered platform that bridges patients and providers can have an outsized real-world impact, especially in resource-limited settings.

2. Objective

Build a full-stack community healthcare web platform where patients can search for doctors, book appointments, view health resources, and manage a basic health profile — while providers can register, manage availability, and view upcoming appointments.

3. Target Users

- Patient / Community Member: Find nearby doctors, book appointments, access health guides, track personal health history.
- Healthcare Provider (Doctor / Clinic): Register a profile, set availability, manage appointment requests, communicate updates.
- Admin / Coordinator: Manage platform listings, approve provider registrations, monitor platform activity.

4. Day 1 Requirements — Frontend Sprint

- Landing Page — hero section with platform mission, call-to-action buttons (Find a Doctor, Book Appointment), and feature highlights. *[5 pts]*
- Doctor/Provider Listing Page — searchable and filterable grid/list of healthcare providers with specialization, rating, and availability badge. *[5 pts]*
- Doctor Profile Page — detailed view showing bio, specialization, location, working hours, and a Book Appointment button. *[4 pts]*
- Appointment Booking Form — multi-step or single form with date/time picker, reason for visit, and patient details. *[5 pts]*
- Patient Dashboard — personal health profile, upcoming appointments, appointment history, and basic vitals input. *[5 pts]*
- Health Resources / Blog Page — paginated list of health articles or tips with category filters (e.g., Nutrition, Mental Health, First Aid). *[4 pts]*
- Registration/Login Pages — separate flows for Patient and Provider with form validation. *[3 pts]*
- 404 / Error State Page — user-friendly error page with navigation options. *[2 pts]*
-

UI/UX Expectations

- Clean, trustworthy aesthetic using a medical color palette (blues, whites, greens).
- Consistent typography hierarchy — headline, subheading, body, caption all clearly differentiated.
- Interactive states on all buttons and links (hover, active, disabled).
- Empty states handled gracefully (e.g., 'No doctors found matching your search').
- Loading skeletons or placeholders on data-heavy sections.

Responsiveness Requirements

- Fully responsive layout tested on mobile (375px), tablet (768px), and desktop (1280px).
- Navigation collapses into a hamburger menu on mobile.
- Booking form is fully usable on touch devices.
- Card grid adapts to single-column on small screens.

Minimum Interactive Functionality

- Live search/filter on the doctor listing page (by name, specialization, or location).
- Date/time picker with disabled past dates on the booking form.
- Form validation with real-time inline error messages (required fields, email format, etc.).
- Toast or notification feedback on form submission.

5. Day 2 Requirements — Backend + Integration

Backend APIs & Logic

- POST /api/auth/register — register a new patient or provider with hashed password, role assignment, and input validation. *[5 pts]*
- POST /api/auth/login — authenticate user, return JWT or session token, role-based redirect. *[5 pts]*
- GET /api/doctors — return paginated list of providers with optional query filters (specialization, city, availability). *[5 pts]*
- GET /api/doctors/:id — return full doctor profile including availability slots. *[4 pts]*
- POST /api/appointments — create a new appointment booking tied to patient and provider, prevent double-booking. *[6 pts]*
- GET /api/appointments/:userId — return appointment history for a patient or schedule for a provider. *[4 pts]*
- PUT /api/appointments/:id — update appointment status (Confirmed / Cancelled / Completed). *[4 pts]*
- GET /api/resources — return paginated health articles with category filtering. *[3 pts]*
-

Database Requirements

- Users table: id, name, email, password (hashed), role (patient/provider/admin), created_at.
- DoctorProfiles table: user_id (FK), specialization, bio, location, working_hours (JSON or text), profile_image_url.
- Appointments table: id, patient_id (FK), doctor_id (FK), datetime, status, reason, created_at.
- HealthResources table: id, title, body, category, author, published_at.
- All tables with appropriate foreign key constraints and indexed fields.

Authentication

- Secure password hashing (bcrypt or equivalent).
- JWT-based or session-based authentication with token expiry.
- Protected routes: patients can only view their own appointments; providers can only manage their own schedule.
- Role-based access control enforced at the API layer.

Frontend–Backend Integration

- Doctor listing page fetches live data from GET /api/doctors with filter parameters.
- Doctor profile page fetches from GET /api/doctors/:id.
- Booking form POSTs to /api/appointments and shows confirmation or error to the user.
- Patient dashboard loads from GET /api/appointments/:userId behind an authenticated session.
- Login/Register forms connect to auth endpoints and store token in localStorage or a cookie.

External APIs (if applicable)

- Optional: Google Maps / Mapbox API to display clinic location on the doctor profile page.
- Optional: A public health API (e.g., Open Disease Data) to display health stats on the landing page.

6. Core Features (Must-Have)

The following features are mandatory and map directly to Functionality & Feature Completion (35%). Omitting any of these will result in a significant scoring penalty.

- Doctor search with at least 2 working filter options (specialization, city, or availability).
- Complete appointment booking flow — form submission persists to the database and is retrievable.
- The patient dashboard shows real appointment data loaded from the backend.
- User registration and login with role differentiation (patient vs. provider).
- Protected routes — unauthenticated users cannot access dashboards or booking.
- Responsive design functioning correctly on mobile and desktop.
- At least 5 seeded doctor profiles visible in the live application.
- Health resources page showing at least 3 articles from the database.

7. Bonus Features — Leaderboard Boosters

Optional features that earn extra leaderboard points. Bonuses are easy to verify and are awarded in order of submission.

- ★ [+10 pts] Real-time appointment status updates using WebSockets or Server-Sent Events.
- ★ [+10 pts] AI-powered symptom checker — user describes symptoms, app suggests relevant specializations.
- ★ [+8 pts] Email or SMS confirmation sent to patient on appointment booking (use SendGrid, Twilio, etc.).
- ★ [+8 pts] Provider dashboard with a calendar view for managing daily schedule.

- ★ [+6 pts] Geolocation-based doctor search — 'Find Doctors Near Me' using browser geolocation API.
- ★ [+6 pts] Dark mode toggle with preference persistence.
- ★ [+5 pts] Patient health vitals tracker (weight, blood pressure, glucose) with a chart visualization.

8. Submission & Scoring Considerations

What Teams Submit

- GitHub repository link (must be public, with a README explaining setup and live URL).
- Deployed application URL accessible without login for initial judge review.
- Brief feature completion checklist submitted through the portal dashboard.

Auto-Checked vs. Judge-Evaluated

- Auto-checked: Page presence, API response codes, form validation, deployed URL accessibility, response time.
- Judge-evaluated: Code architecture, design polish, innovation, pitch delivery, scalability approach.

Suggested Scoring Breakdown

Criterion	Points	Method
Doctor search + listing page (live data)	10	Auto-checked
Appointment booking (DB persistence)	12	Auto + Manual
Patient dashboard with real data	8	Auto-checked
Authentication (register + login + protect)	10	Manual
API design quality & error handling	10	Judge-scored
UI/UX polish & responsiveness	12	Judge-scored
Final pitch (clarity, demo, Q&A)	8	Judge-scored
Bonus features (each item scored separately)	Up to +48	Auto + Manual

9. Technical Constraints

- Any modern full-stack framework is permitted.
- AI tools and assistants are allowed and encouraged for productivity.
- App must be live on a public URL for judge evaluation (Vercel, Netlify, Railway, etc.).
- No local-only setups; everything must run in-browser without installation.
- GitHub repository must be public and submitted through the portal.

10. Evaluation Mapping

- Core booking + doctor search flow → Functionality & Feature Completion (35%)
- REST API design, error handling, DB schema → Backend Architecture & API Design (20%)
- AI symptom checker, real-time updates → Innovation & Scalability (15%)
- Color system, typography, mobile layout → UI/UX & Responsiveness (12%)
- Code organization, naming, comments → Code Quality (10%)
- 2-3 min demo highlighting user journey → Final Pitch (8%)

AI-POWERED TASK AUTOMATION TOOL

1. Problem Statement

Professionals and students waste significant time on repetitive task management, scheduling conflicts, and context-switching between tools. Existing productivity apps are either too complex or offer no intelligence — they simply store tasks without helping users prioritize, plan, or automate recurring workflows. There is a clear gap for a smart, AI-driven assistant that can understand natural language, suggest actions, and automate routine cognitive work.

Why It Matters

Research consistently shows that context-switching and poor task prioritization are among the top causes of lost productivity. An accessible AI productivity tool can dramatically reduce cognitive load, improve time management, and help users achieve their goals with less friction — applicable for students, freelancers, developers, and teams alike.

2. Objective

Build an AI-powered productivity web application that allows users to manage tasks, schedule events, and automate workflows through a combination of a structured UI and a natural language chatbot interface. The AI should interpret user intent, suggest priorities, and generate schedules.

3. Target Users

- Student / Learner: Manage assignments, set study schedules, get reminders, ask the AI to plan their week.
- Professional / Freelancer: Track projects, set deadlines, automate recurring tasks, brief AI on priorities.
- Team Lead / Manager: Assign tasks, monitor progress, use AI to summarize status and suggest reallocation.

4. Day 1 Requirements — Frontend Sprint

- Landing Page — value proposition, feature highlights, a live demo prompt input, and a 'Get Started' CTA. *[5 pts]*
- User Dashboard — overview cards showing pending tasks, completed tasks, upcoming events, and AI-generated daily summary. *[6 pts]*

- Task Manager View — list/kanban view of tasks with priority tags, due dates, status toggles, and add/edit/delete functionality. *[6 pts]*
- AI Chat Interface Page — chat-style UI where users type natural language commands (e.g., 'Add a task: finish report by Friday, high priority'). *[6 pts]*
- Calendar / Schedule View — weekly or monthly view of scheduled tasks and events with color coding by priority. *[5 pts]*
- Settings / Preferences Page — notification preferences, working hours, time zone, and theme toggle. *[3 pts]*
- Onboarding / Welcome Flow — brief guided setup for first-time users (name, role, productivity goals). *[3 pts]*

UI/UX Expectations

- Modern, distraction-free UI — minimal color palette, generous whitespace, clear focus indicators.
- Chat interface with message bubbles, typing indicator, and timestamp.
- Kanban board (if used) with drag-and-drop visual interaction (even if drag is not yet functional on Day 1).
- Priority color coding system consistently applied: red = high, amber = medium, green = low.
- Clear empty states for the task list and calendar.

Responsiveness Requirements

- Dashboard collapses cleanly to a mobile view with bottom navigation or hamburger menu.
- Chat interface is fully usable on mobile touch screens.
- Task cards display correctly from 320px to 1920px viewport width.
- Calendar view gracefully degrades to a list/agenda view on small screens.

Minimum Interactive Functionality

- Add/edit/delete task from the task manager UI (frontend state only on Day 1 is acceptable).
- Priority and status filters working on the task list.
- Chat input sends a message and displays a mock AI response (streaming or static).
- Calendar day cells are clickable to view tasks for that date.

5. Day 2 Requirements — Backend + Integration

Backend APIs & Logic

- POST `/api/auth/register` and POST `/api/auth/login` — user auth with JWT, hashed passwords. *[5 pts]*
- GET/POST/PUT/DELETE `/api/tasks` — full CRUD for tasks with fields: title, description, due_date, priority, status, user_id. *[8 pts]*
- POST `/api/ai/chat` — accepts a user message, sends it to an AI API (OpenAI / Gemini / Anthropic), returns the AI response. Must parse actionable commands (e.g., task creation from chat). *[10 pts]*
- POST `/api/ai/schedule` — AI endpoint that accepts user tasks and returns an AI-generated optimized daily schedule. *[8 pts]*
- GET `/api/tasks/summary` — returns a JSON summary (counts by status, overdue tasks, priority breakdown) for dashboard cards. *[5 pts]*

- GET /api/events/calendar — returns tasks formatted as calendar events (with start/end times) for the calendar view. [4 pts]

Database Requirements

- Users table: id, name, email, password, timezone, role, created_at.
- Tasks table: id, user_id (FK), title, description, due_date, priority (low/medium/high), status (todo/in-progress/done), created_at.
- Events table: id, user_id (FK), title, start_datetime, end_datetime, linked_task_id (nullable).
- ChatHistory table: id, user_id (FK), role (user/assistant), message, timestamp — to maintain conversation context.
- Proper indexing on user_id and due_date columns.

Authentication

- JWT-based authentication with refresh token or session expiry.
- All task and chat routes require authenticated user context.
- User-scoped data isolation — users can only access their own tasks and chat history.

Frontend–Backend Integration

- Task manager CRUD operations fully connected to the backend API.
- Chat messages are sent to POST /api/ai/chat; responses rendered in the chat UI.
- When the AI parses a task creation command, the task is automatically created in the database and appears in the task list.
- Dashboard summary cards load from GET /api/tasks/summary.
- Calendar view loads events from GET /api/events/calendar.

External APIs (if applicable)

- Required: Integration with at least one AI API — OpenAI (gpt-4o-mini), Google Gemini, or Anthropic Claude.
- Optional: Natural Language Processing library (compromise.js, duckling) for offline intent parsing as a fallback.
- Optional: Google Calendar API for two-way sync of events.

6. Core Features (Must-Have)

The following features are mandatory and map directly to Functionality & Feature Completion (35%). Omitting any of these will result in a significant scoring penalty.

- Full task CRUD — create, read, update, delete tasks from the UI, persisted to the database.
- AI chat interface connected to a real AI API — responses are contextually relevant to user tasks.
- AI-generated daily schedule — at least one working AI schedule endpoint that reads user tasks and returns a plan.
- Task filtering by priority and status, working end-to-end.
- User authentication — registration and login required to access the dashboard.
- Calendar view displaying real task data as events.
- Dashboard summary cards showing live task statistics.

7. Bonus Features — Leaderboard Boosters

Optional features that earn extra leaderboard points. Bonuses are easy to verify and are awarded in order of submission.

- ★ [+12 pts] AI agent that reads all tasks and autonomously reorganizes priorities when a new high-urgency task is added.
- ★ [+10 pts] Real-time collaboration — multiple users can share a task board and see live updates via WebSockets.
- ★ [+8 pts] Voice input on the chat interface using the Web Speech API.
- ★ [+8 pts] Recurring task automation — user defines a rule, system auto-generates tasks on schedule.
- ★ [+6 pts] Productivity analytics page — charts showing task completion rate, average time-to-completion, daily output trends.
- ★ [+6 pts] Browser push notifications for upcoming task deadlines.
- ★ [+5 pts] Dark mode with user preference persisted in the database.

8. Submission & Scoring Considerations

What Teams Submit

- GitHub repository link (public, with README listing the AI API used, setup instructions, and live URL).
- Deployed application URL — must include at least 3 seeded tasks for demo purposes.
- Feature completion checklist submitted through the portal.

Auto-Checked vs. Judge-Evaluated

- Auto-checked: Page presence, API response codes, form validation, deployed URL accessibility, response time.
- Judge-evaluated: Code architecture, design polish, innovation, pitch delivery, scalability approach.

Suggested Scoring Breakdown

Criterion	Points	Method
Task CRUD (full, DB-persisted)	12	Auto-checked
AI chat integration (real API responses)	14	Auto + Manual
AI schedule generation endpoint	10	Manual
Calendar view with real data	6	Auto-checked
Auth + protected routes	8	Manual
UI/UX polish & responsiveness	12	Judge-scored
Final pitch (demo + Q&A)	8	Judge-scored

Criterion	Points	Method
Bonus features (each scored separately)	Up to +55	Auto + Manual

9. Technical Constraints

- Any modern full-stack framework is permitted.
- AI tools and assistants are allowed and encouraged for productivity.
- App must be live on a public URL for judge evaluation (Vercel, Netlify, Railway, etc.).
- No local-only setups; everything must run in-browser without installation.
- GitHub repository must be public and submitted through the portal.

10. Evaluation Mapping

- Task CRUD, dashboard summary, calendar data → Functionality & Feature Completion (35%)
- AI API integration, schema design, REST structure → Backend Architecture & API Design (20%)
- AI agent behavior, real-time collab, voice input → Innovation & Scalability (15%)
- Chat UI design, responsive layout, animations → UI/UX & Responsiveness (12%)
- Modular code, prompt engineering quality → Code Quality (10%)
- Live demo of AI interaction, articulate pitch → Final Pitch (8%)

FINANCIAL DATA DASHBOARD

1. Problem Statement

Individual investors and finance students lack accessible, clean tools to visualize market data, track portfolio performance, and make sense of complex financial trends. Commercial tools are either paywalled, overwhelmingly complex, or cluttered with advertising. There is a strong demand for a streamlined, data-rich financial dashboard that surfaces actionable insights through clear visualizations.

Why It Matters

Financial literacy and access to market data directly affect personal wealth-building outcomes. A well-designed dashboard that makes data digestible can empower retail investors to make better decisions, help students learn market dynamics, and bridge the gap between raw financial data and actionable intelligence.

2. Objective

Build a full-stack financial data dashboard that displays real-time or near-real-time stock market data via public APIs, allows users to build and track a personal portfolio, visualizes historical performance through interactive charts, and surfaces key financial metrics.

3. Target Users

- Retail Investor: Track owned stocks, visualize performance, set price alerts, compare assets.
- Finance Student: Explore market data, understand trends, compare sectors, save a watchlist.
- Analyst / Researcher: Deep-dive into individual stock metrics, export data, view sector breakdowns.

4. Day 1 Requirements — Frontend Sprint

- Landing Page — market summary ticker bar, top-gaining/losing stocks, platform feature highlights, and CTA to sign in. *[5 pts]*
- Market Overview Page — sector performance grid, top movers list, and a global market indices table (S&P 500, NASDAQ, DOW, NIFTY, etc.). *[6 pts]*
- Stock Search & Detail Page — search by ticker symbol, display company overview, current price, daily change, key metrics (P/E, Market Cap, Volume), and a placeholder chart. *[6 pts]*
- Portfolio Tracker Page — user-created portfolio with buy price, current price, gain/loss per asset, and portfolio total performance. *[6 pts]*
- Watchlist Page — saved stocks list with real-time price display and quick-add functionality. *[4 pts]*
- Charts & Analytics Page — interactive historical price chart (line/candlestick) for a selected stock or index. *[5 pts]*
- User Profile / Settings Page — display name, connected API key display, currency preference, notification settings. *[2 pts]*

UI/UX Expectations

- Financial data aesthetic: dark or light mode with a professional data-dense layout (think Bloomberg-lite).
- Color coding for market direction: green for gains, red for losses, consistent throughout all pages.
- Interactive chart with hover tooltips showing exact price/date values.
- Data tables with sortable columns for market overview and portfolio pages.
- Loading states and refresh indicators on data-heavy components.

Responsiveness Requirements

- Charts resize fluidly across viewport widths — no horizontal scrolling on mobile.
- Data tables become horizontally scrollable on small screens rather than collapsing awkwardly.
- Portfolio page cards stack vertically on mobile.
- Market ticker bar runs as a smooth horizontal scroll animation on all screen sizes.

Minimum Interactive Functionality

- Live stock ticker animation (simulated or live) in the header or landing page.
- Search bar with debounced auto-suggest showing ticker symbols.
- Chart time-range selector (1D / 1W / 1M / 3M / 1Y) — updates chart visualization.
- Add to Watchlist button — adds stock to the watchlist (frontend state on Day 1).

5. Day 2 Requirements — Backend + Integration

Backend APIs & Logic

- GET /api/market/quote/:symbol — fetches current stock price, change, volume, market cap from a public API (Alpha Vantage, Yahoo Finance, Finnhub, Polygon.io). *[8 pts]*
- GET /api/market/history/:symbol?range=1M — fetches historical OHLCV data for a given symbol and range, formatted for chart rendering. *[8 pts]*
- GET /api/market/overview — returns market indices and top movers from the external API, cached server-side (TTL: 60 seconds). *[6 pts]*
- GET/POST/DELETE /api/portfolio — full CRUD for user portfolio entries: add stock (symbol, quantity, buy_price), delete holding, compute current value. *[8 pts]*
- GET/POST/DELETE /api/watchlist — add/remove/list watchlist stocks for authenticated user. *[5 pts]*
- POST /api/auth/register and POST /api/auth/login — user authentication with JWT. *[5 pts]*

Database Requirements

- Users table: id, email, password (hashed), display_name, currency_preference, created_at.
- Portfolio table: id, user_id (FK), symbol, company_name, quantity, buy_price, buy_date.
- Watchlist table: id, user_id (FK), symbol, added_at.
- PriceCache table (optional but scored): symbol, data (JSON), cached_at — reduces external API calls.
- All user-owned data scoped and indexed by user_id.

Authentication

- JWT authentication on all portfolio and watchlist endpoints.
- API key for external financial data stored as a server-side environment variable (never exposed to the client).
- Rate-limit-aware backend — implement server-side caching to avoid exceeding free-tier API limits during demos.

Frontend–Backend Integration

- Stock detail page fetches live quote from GET /api/market/quote/:symbol.
- Chart component receives data from GET /api/market/history/:symbol and renders using a chart library (Chart.js, Recharts, Lightweight Charts, etc.).
- Portfolio page POSTs new holdings to /api/portfolio and loads the full list on mount, computing P&L from live quote data.
- Watchlist page syncs with /api/watchlist and displays current prices from cached quotes.
- Market overview page loads from GET /api/market/overview with visible refresh indicator.

External APIs (if applicable)

- Required: At least one public financial API integrated on the backend — Alpha Vantage (free tier), Finnhub, Polygon.io, or Yahoo Finance (unofficial).
- Optional: CoinGecko API for a Crypto section.
- Optional: NewsAPI.org for a financial news feed widget.

6. Core Features (Must-Have)

The following features are mandatory and map directly to Functionality & Feature Completion (35%). Omitting any of these will result in a significant scoring penalty.

- ☐ Live stock price lookup working end-to-end via an external API.
- ☐ Historical price chart rendering with real data for at least one time range.
- ☐ Portfolio CRUD — add, view, and remove holdings, with computed gain/loss.
- ☐ Watchlist — save and retrieve at least 5 stocks, with live prices.
- ☐ User authentication — login required to access portfolio and watchlist.
- ☐ Market overview page showing at least 3 major indices and top movers.
- ☐ Server-side caching or rate-limiting logic to handle API quota constraints.
- ☐ Responsive layout working on mobile and desktop.

7. Bonus Features — Leaderboard Boosters

Optional features that earn extra leaderboard points. Bonuses are easy to verify and are awarded in order of submission.

- ★ [+12 pts] AI-powered stock analysis assistant — user enters a ticker, AI provides a brief technical/fundamental commentary using current data.
- ★ [+10 pts] Live price streaming using WebSockets (e.g., Finnhub WebSocket feed) with auto-updating chart.

- ★ [+8 pts] Portfolio performance analytics — pie chart by sector, total return over time, Sharpe ratio approximation.
- ★ [+8 pts] Price alert system — user sets a target price, backend polls and sends an in-app or email notification when triggered.
- ★ [+6 pts] Crypto integration — unified dashboard showing both stocks and cryptocurrencies via CoinGecko.
- ★ [+6 pts] CSV/PDF portfolio export.
- ★ [+5 pts] Candlestick chart implementation (more complex than a line chart).

8. Submission & Scoring Considerations

What Teams Submit

- GitHub repository link — README must specify which financial API is used and how to set up the API key.
- Deployed application URL — seeded with at least 3 demo portfolio holdings.
- Feature completion checklist submitted through the portal.

Auto-Checked vs. Judge-Evaluated

- Auto-checked: Page presence, API response codes, form validation, deployed URL accessibility, response time.
- Judge-evaluated: Code architecture, design polish, innovation, pitch delivery, scalability approach.

Suggested Scoring Breakdown

Criterion	Points	Method
Live stock quote API integration	12	Auto-checked
Historical chart with real data	10	Auto + Manual
Portfolio CRUD + P&L calculation	12	Auto-checked
Watchlist with live prices	6	Auto-checked
Auth + API key security	8	Manual
UI/UX polish & data clarity	12	Judge-scored
Final pitch (demo + Q&A)	8	Judge-scored
Bonus features (each scored separately)	Up to +55	Auto + Manual

9. Technical Constraints

- Any modern full-stack framework is permitted.
- AI tools and assistants are allowed and encouraged for productivity.
- App must be live on a public URL for judge evaluation (Vercel, Netlify, Railway, etc.).
- No local-only setups; everything must run in-browser without installation.

- GitHub repository must be public and submitted through the portal.

10. Evaluation Mapping

- Live API data, portfolio tracker, watchlist → Functionality & Feature Completion (35%)
- Caching layer, API abstraction, schema design → Backend Architecture & API Design (20%)
- AI stock analysis, live WebSocket streaming → Innovation & Scalability (15%)
- Chart design, data density, color system → UI/UX & Responsiveness (12%)
- Clean API layer, environment variable handling → Code Quality (10%)
- Clear demo of live data + portfolio value → Final Pitch (8%)

SUSTAINABILITY & GREEN TECH PLATFORM

1. Problem Statement

Despite growing awareness of climate change, most individuals have no practical tool to measure, track, or reduce their personal or organizational environmental impact. Waste continues to be poorly sorted, energy consumption goes unmonitored, and carbon footprints remain abstract figures. Communities lack accessible platforms that turn environmental data into concrete, motivating actions.

Why It Matters

Individual behavior change at scale is one of the most powerful levers for sustainability. A platform that makes environmental data personal, social, and actionable — through gamification, community challenges, and AI-driven recommendations — can drive measurable reductions in carbon emissions, waste output, and energy consumption. Technology has a critical role to play in the green transition.

2. Objective

Build a full-stack sustainability web platform that allows users to track their personal carbon footprint, log sustainable actions, join community challenges, monitor energy/waste metrics, and receive AI-powered recommendations for greener lifestyle choices.

3. Target Users

- Eco-Conscious Individual: Track daily carbon footprint, log sustainable actions (recycling, cycling), get personalized tips.
- Community Organizer / NGO: Create and manage green challenges, track community participation, publish environmental data.
- Student / Educator: Explore environmental dashboards, compare regional climate data, share sustainability reports.

4. Day 1 Requirements — Frontend Sprint

- Landing Page — environmental impact stats, platform mission, community challenge previews, and a carbon footprint calculator entry point. *[5 pts]*
- Personal Dashboard — user's daily carbon footprint score, streak tracker, green action log, and level/badge display. *[6 pts]*
- Carbon Footprint Calculator Page — interactive form with categories (transport, diet, energy, shopping) that calculates and visualizes the user's CO2 footprint. *[7 pts]*
- Community Challenges Page — list of active sustainability challenges with participation count, progress bars, and join/leave buttons. *[5 pts]*
- Eco-Action Log Page — users log sustainable actions (e.g., 'took the bus', 'recycled 2kg', 'planted a tree'), each with a points value and category. *[5 pts]*
- Environmental Data Map / Regional Stats Page — visual display of pollution, air quality, or energy data by region using a map or chart. *[5 pts]*

- Leaderboard Page — community ranking by total eco-points, filterable by week/month/all-time. [4 pts]
- Registration / Login Pages — standard auth UI with optional 'sustainability profile' setup (city, transport habits, diet type). [3 pts]

UI/UX Expectations

- Nature-inspired design language: earthy greens, sky blues, clean whites — avoid overly corporate look.
- Gamification elements visible throughout: progress bars, badge icons, streak counters, XP displays.
- Carbon footprint calculator with animated feedback when the score updates.
- Data visualizations (charts, gauges, maps) using a consistent color-to-impact scale.
- Motivational microcopy throughout (e.g., 'You saved 3.2kg CO2 today — that is equivalent to planting 0.3 trees').

Responsiveness Requirements

- Dashboard and calculator fully functional on mobile — form inputs are touch-friendly.
- Community challenge cards stack to single column on mobile.
- Leaderboard scrolls vertically on small screens without truncating ranks.
- Map/chart components scale responsively without overflowing containers.

Minimum Interactive Functionality

- Carbon calculator dynamically updates the footprint estimate as user changes form inputs (real-time calculation in-browser).
- Action log form with category selector and points preview before submission.
- Challenge progress bar animates when the page loads.
- Leaderboard filters (weekly/monthly/all-time) update the displayed list.

5. Day 2 Requirements — Backend + Integration

Backend APIs & Logic

- POST /api/auth/register and POST /api/auth/login — user auth with JWT, sustainability profile fields captured at registration. [5 pts]
- POST /api/footprint/calculate — accepts transport, energy, diet, and shopping inputs; runs server-side calculation using verified emission factors; returns CO2 breakdown by category. [10 pts]
- GET/POST /api/actions — log a sustainable action (category, description, co2_saved, points_earned); retrieve action history for a user. [8 pts]
- GET/POST/PUT /api/challenges — create a community challenge (title, description, target, end_date); join/leave a challenge; return participation stats. [8 pts]
- GET /api/leaderboard — return top users ranked by total eco-points, with weekly/monthly/all-time filter. [5 pts]
- GET /api/env/airquality?city=Lahore — proxy query to a public environmental API (OpenAQ, WAQI, Open-Meteo) and return AQI and pollution data for a given city. [6 pts]

Database Requirements

- Users table: id, name, email, password, city, diet_type, transport_habits, total_eco_points, streak_days, created_at.
- FootprintLogs table: id, user_id (FK), transport_co2, energy_co2, diet_co2, shopping_co2, total_co2, logged_date.
- EcoActions table: id, user_id (FK), category, description, co2_saved, points_earned, logged_at.
- Challenges table: id, creator_id (FK), title, description, target_metric, end_date, participant_count.
- ChallengeParticipants table: id, challenge_id (FK), user_id (FK), joined_at, contribution_value.
- Proper foreign key constraints; index on user_id and logged_date for performance.

Authentication

- JWT authentication required for all footprint, action, and challenge endpoints.
- User-scoped data isolation — footprint logs and eco-actions are private unless shared.
- Public endpoints: leaderboard (read-only) and challenge listing are accessible without login.

Frontend–Backend Integration

- Carbon calculator POSTs to /api/footprint/calculate and renders the server-computed breakdown in the pie chart.
- Eco-action log submits to POST /api/actions; history loads from GET /api/actions on page mount.
- Challenges page loads from GET /api/challenges; join button sends a POST to /api/challenges/:id/join.
- Leaderboard loads from GET /api/leaderboard with active filter applied.
- Environmental map/widget fetches from GET /api/env/airquality with user's city auto-populated.

External APIs (if applicable)

- Required: At least one public environmental API — OpenAQ (air quality), Open-Meteo (weather/climate), or WAQI (AQI data).
- Optional: Carbon Interface API for detailed carbon emission factor calculations.
- Optional: NASA EONET API for natural event/disaster data overlay on the map.

6. Core Features (Must-Have)

The following features are mandatory and map directly to Functionality & Feature Completion (35%). Omitting any of these will result in a significant scoring penalty.

- ☐ Carbon footprint calculator with at least 3 input categories, server-side calculation, and visual breakdown.
- ☐ Eco-action log — add, view, and track actions, with points system updating the user's score.
- ☐ Community challenges — at least 2 active challenges visible; join/leave functionality working end-to-end.
- ☐ Leaderboard showing at least 5 users ranked by eco-points, with working time filters.
- ☐ Live environmental data from an external API displayed on the platform (air quality or climate metric).

- ☐ User authentication with a sustainability profile (city, habits).
- ☐ Streak tracker on the dashboard updating based on daily login or action logging.
- ☐ Responsive design working correctly on mobile and desktop.

7. Bonus Features — Leaderboard Boosters

Optional features that earn extra leaderboard points. Bonuses are easy to verify and are awarded in order of submission.

- ★ [+12 pts] AI sustainability advisor — user describes their lifestyle, AI generates a personalized weekly green action plan with estimated CO2 savings.
- ★ [+10 pts] Social sharing — users can share their eco-score card or a completed challenge to social media (Open Graph meta tags).
- ★ [+8 pts] Interactive map showing regional AQI or carbon intensity data with color-coded markers (Leaflet.js or Mapbox).
- ★ [+8 pts] Team/Group challenges — organizations or friend groups compete in a shared challenge with a group leaderboard.
- ★ [+6 pts] Monthly sustainability report — PDF or in-app view showing the user's footprint trend, top actions, and CO2 saved vs. previous month.
- ★ [+6 pts] Gamification tier system — Bronze/Silver/Gold/Platinum eco-levels with unlockable badges and milestone animations.
- ★ [+5 pts] Offline-capable PWA — service worker caching for core pages so the app works without network.

8. Submission & Scoring Considerations

What Teams Submit

- GitHub repository link — README must specify which environmental API is used and include setup instructions.
- Deployed application URL — seeded with at least 5 eco-actions and 2 active challenges for demo purposes.
- Feature completion checklist submitted through the portal.

Auto-Checked vs. Judge-Evaluated

- Auto-checked: Page presence, API response codes, form validation, deployed URL accessibility, response time.
- Judge-evaluated: Code architecture, design polish, innovation, pitch delivery, scalability approach.

Suggested Scoring Breakdown

Criterion	Points	Method
Carbon calculator (server-side, visual breakdown)	12	Auto + Manual
Eco-action log (CRUD + points system)	10	Auto-checked

Criterion	Points	Method
Community challenges (join/leave, DB-persisted)	10	Auto-checked
Leaderboard with time filters	6	Auto-checked
External environmental API integration	8	Manual
UI/UX polish & gamification elements	12	Judge-scored
Final pitch (demo + Q&A)	8	Judge-scored
Bonus features (each scored separately)	Up to +55	Auto + Manual

9. Technical Constraints

- Any modern full-stack framework is permitted.
- AI tools and assistants are allowed and encouraged for productivity.
- App must be live on a public URL for judge evaluation (Vercel, Netlify, Railway, etc.).
- No local-only setups; everything must run in-browser without installation.
- GitHub repository must be public and submitted through the portal.

10. Evaluation Mapping

- Calculator, action log, challenges, leaderboard → Functionality & Feature Completion (35%)
- Emission factor logic, API proxy, DB schema → Backend Architecture & API Design (20%)
- AI advisor, social sharing, team challenges, PWA → Innovation & Scalability (15%)
- Nature-inspired design, gamification UI, maps → UI/UX & Responsiveness (12%)
- Modular emission calculation, clean data models → Code Quality (10%)
- Storytelling around environmental impact and demo → Final Pitch (8%)

